

Graph Representation

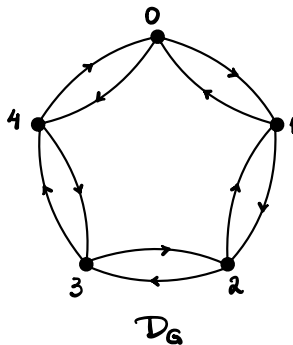
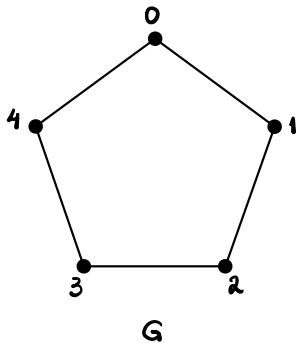
A graph $G=(V,E)$ is usually represented as its **symmetric digraph**

$$\mathcal{D}_G := (V, \bigcup_{uv \in E} \{(u,v), (v,u)\}).$$

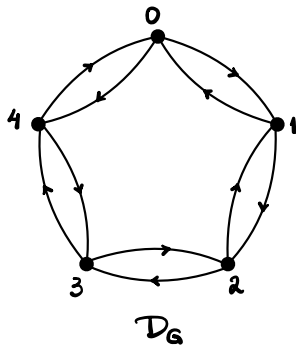
syntactic sugar for $\bigcup \{ \{(u,v), (v,u)\} : uv \in E \}$

also applies for $\cap, \Sigma, \Pi, \max$, and many others

Example



~~Typical Representation with Adjacency Lists~~ Arrays



Adj[0]

	1	4
	0	2
	1	3
	2	4
	0	3

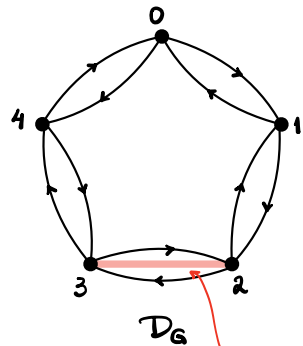
⋮

Adj[4]

2
2
2
2
2

sizes

~~Typical Representation with Adjacency Lists~~ Arrays



Adj[0]

⋮

Adj[4]

	1	4
	0	2
	1	3
	2	4
	0	3

sizes

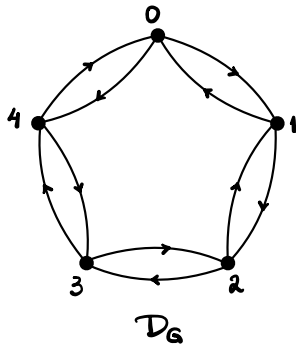
2
2
2
2
2

where do we store an attribute of

this EDGE, e.g. $c(\{2,3\})$?

↑
cost

~~Typical Representation with Adjacency Lists~~ Arrays



Adj[0]

	1	4
	0	2
	1	3
	2	4
	0	3

⋮

Adj[4]

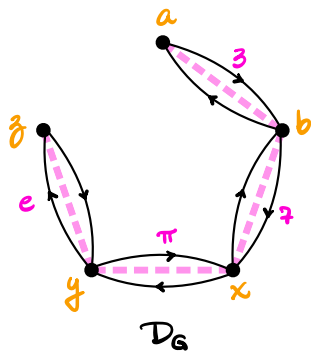
2
2
2
2
2

sizes

We may have **multiple arrays**, each indexed (in constant time) by **vxs** or **arcs/edges**, providing access to each attribute.

E.g., **cost**[v], **weight**[a], and so on

Typical Representation with Adjacency ~~Lists~~ Arrays



Adj[a]		3	
Adj[b]		3	7
Adj[x]		π	7
Adj[y]		e	π
Adj[z]		e	

1
2
2
2
1

sizes

ends[3]	a	b
ends[π]	x	y
ends[e]	y	z
ends[7]	b	x

cost[3]	6
cost[π]	9
cost[e]	5
cost[7]	$\frac{1}{2}$

We may have multiple arrays, each indexed (in constant time) by vxs or arcs/edges, providing access to each attribute.

E.g., cost[v], weight[a], and so on

Matching Verification

Recall: $M \subseteq E(G)$ is a **matching** of a graph G if M is pairwise disjoint

VERIFY-MATCHING(G, M)

↑
given how?

Matching Verification

Recall: $M \subseteq E(G)$ is a **matching** of a graph G if M is pairwise disjoint

VERIFY-MATCHING(G, M)

$\uparrow$
given as a $\{0,1\}$ -valued vector indexed by E ,

e.g., for each $e \in E$, one has $M[e] := \begin{cases} 1 & \text{if } e \in M; \\ 0 & \text{otherwise.} \end{cases}$

Matching Verification

Recall: $M \subseteq E(G)$ is a **matching** of a graph G if M is pairwise disjoint

VERIFY-MATCHING(G, M)

↑
given as a $\{0,1\}$ -valued vector indexed by E ,

e.g., for each $e \in E$, one has $M[e] := \begin{cases} 1 & \text{if } e \in M; \\ 0 & \text{otherwise.} \end{cases}$

this is so ubiquitous that we have a special notation for it,

called the **Iverson bracket**: for each predicate P ,

an expression that evaluates to true or false

denote $[P] := \begin{cases} 1 & \text{if } P \text{ holds;} \\ 0 & \text{otherwise.} \end{cases}$

Matching Verification

Recall: $M \subseteq E(G)$ is a **matching** of a graph G if M is pairwise disjoint

VERIFY-MATCHING(G, M)

1. Zero initialize a vector `matched` indexed by V
2. **for each** $e \in E(G)$ **do**
3. **if** $M[e] = 1$ **then**
4. $\{u, v\} \leftarrow e$
5. **if** $\text{matched}[u] = 1$ **or** $\text{matched}[v] = 1$ **then return FALSE**
6. $\text{matched}[u] \leftarrow \text{matched}[v] \leftarrow 1$
7. **return TRUE**

Matching Verification

Recall: $M \subseteq E(G)$ is a **matching** of a graph G if M is pairwise disjoint

VERIFY-MATCHING(G, M)

1. Zero initialize a vector `matched` indexed by V
2. **for each** $e \in E(G)$ **do** may also be written as ' $e \in M$ '
3. **if** $M[e] = 1$ **then**
4. $\{u, v\} \leftarrow e$
5. **if** `matched` $[u] = 1$ **or** `matched` $[v] = 1$ **then return FALSE**
6. `matched` $[u] \leftarrow$ `matched` $[v] \leftarrow 1$
7. **return TRUE**

Matching Verification

Recall: $M \subseteq E(G)$ is a **matching** of a graph G if M is pairwise disjoint

VERIFY-MATCHING(G, M)

1. Zero initialize a vector `matched` indexed by V
2. **for each** $e \in E(G)$ **do**
3. **if** $e \in M$ **then**
4. $\{u, v\} \leftarrow e$
5. **if** `matched`[u] = 1 **or** `matched`[v] = 1 **then return FALSE**
6. `matched`[u] $\leftarrow$ `matched`[v] $\leftarrow$ 1
7. **return TRUE**

Matching Verification

Recall: $M \subseteq E(G)$ is a **matching** of a graph G if M is pairwise disjoint

VERIFY-MATCHING(G, M)

1. Zero initialize a vector **matched** indexed by V
2. for each $e \in E(G)$ do
3. if $e \in M$ then
4. $\{u, v\} \leftarrow e$
5. if $\text{matched}[u] = 1$ or $\text{matched}[v] = 1$ then return FALSE
6. $\text{matched}[u] \leftarrow \text{matched}[v] \leftarrow 1$
7. return TRUE

this vector will ultimately represent

$$\text{the set } V_M := \bigcup M = \bigcup_{e \in M} e$$

Matching Verification

Recall: $M \subseteq E(G)$ is a **matching** of a graph G if M is pairwise disjoint

VERIFY-MATCHING(G, M)

1. $V_M \leftarrow \emptyset$ $\triangleright V_M \subseteq V(G)$ type hint

2. for each $e \in E(G)$ do

3. if $e \in M$ then

4. $\{u, v\} \leftarrow e$

5. if $u \in V_M$ or $v \in V_M$ then return FALSE

6. $V_M \leftarrow V_M \cup \{u, v\}$

7. return TRUE

Matching Verification

Recall: $M \subseteq E(G)$ is a **matching** of a graph G if M is pairwise disjoint

VERIFY-MATCHING(G, M)

1. $V_M \leftarrow \emptyset$ $\triangleright V_M \subseteq V(G)$ type hint
2. for each $e \in E(G)$ do
3. if $e \in M$ then
4. $\{u, v\} \leftarrow e$
5. if $u \in V_M$ or $v \in V_M$ then return FALSE
6. $V_M \leftarrow V_M \cup \{u, v\}$
7. return TRUE

Matching Verification

Recall: $M \subseteq E(G)$ is a **matching** of a graph G if M is pairwise disjoint

VERIFY-MATCHING(G, M)

1. $V_M \leftarrow \emptyset$ $\triangleright V_M \subseteq V(G)$ type hint

2. for each $e \in E(G)$ do
3. if $e \in M$ then

} AWKWARD

if convenient, we may assume that M is ALSO given as a linked list, so one may iterate over edges of M in time $O(|M|)$

4. $\{u, v\} \leftarrow e$

5. if $u \in V_M$ or $v \in V_M$ then return FALSE

6. $V_M \leftarrow V_M \cup \{u, v\}$

7. return TRUE

Matching Verification

Recall: $M \subseteq E(G)$ is a **matching** of a graph G if M is pairwise disjoint

VERIFY-MATCHING(G, M)

1. $V_M \leftarrow \emptyset$ $\triangleright V_M \subseteq V(G)$ type hint
2. for each $e \in E(G)$ do
3. if $e \in M$ then
4. $\{u, v\} \leftarrow e$
5. if $u \in V_M$ or $v \in V_M$ then return FALSE
6. $V_M \leftarrow V_M \cup \{u, v\}$
7. return TRUE

AWKWARD

if convenient, we may assume that M is ALSO given as a linked list, so one may iterate over edges of M in time $O(|M|)$

OK for subsets of vxs or edges/arcs

Matching Terminology and Notation

Let $G := (V, E)$ be a graph and let $M \subseteq E$ be a *matching* of G .

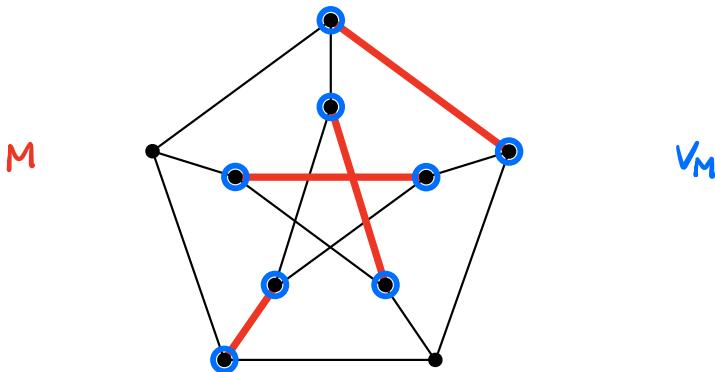
We denote $V_M := \bigcup_{e \in M} e$.

Matching Terminology and Notation

Let $G := (V, E)$ be a graph and let $M \subseteq E$ be a *matching* of G .

We denote $V_M := \bigcup_{e \in M} e$.

Example



Matching Terminology and Notation

Let $G := (V, E)$ be a graph and let $M \subseteq E$ be a **matching** of G .

We denote $V_M := \bigcup_{e \in M} e$.

For each $u \in V_M$, we say that M **covers** or **saturates** u ,

and we call u **M -covered** or **M -saturated**.

Matching Terminology and Notation

Let $G := (V, E)$ be a graph and let $M \subseteq E$ be a **matching** of G .

We denote $V_M := \bigcup_{e \in M} e$.

For each $u \in V_M$, we say that M **covers** or **saturates** u ,
and we call u **M -covered** or **M -saturated**.

For each $u \in V \setminus V_M$, we say that M **misses** or **exposes** u
and we call u **M -free** or **M -exposed**.

Matching Terminology and Notation

Let $G := (V, E)$ be a graph and let $M \subseteq E$ be a **matching** of G .

We denote $V_M := \bigcup_{e \in M} e$.

For each $u \in V_M$, we say that M **covers** or **saturates** u ,
and we call u **M -covered** or **M -saturated**.

For each $u \in V \setminus V_M$, we say that M **misses** or **exposes** u
and we call u **M -free** or **M -exposed**.

The matching M is called **perfect** if $V_M = V$.

Perfect Matching Verification

VERIFY-PERFECT-MATCHING(G, M)

1. $V_M \leftarrow \emptyset$ $\triangleright V_M \subseteq V(G)$
2. for each $e \in E(G)$ do
3. if $e \in M$ then
4. $\{u, v\} \leftarrow e$
5. if $u \in V_M$ or $v \in V_M$ then return FALSE
6. $V_M \leftarrow V_M \cup \{u, v\}$
7. return $V_M = V(G)$

Perfect Matching Verification

VERIFY-PERFECT-MATCHING(G, M)

1. $V_M \leftarrow \emptyset$ $\triangleright V_M \subseteq V(G)$

2. for each $e \in E(G)$ do

3. if $e \in M$ then

4. $\{u, v\} \leftarrow e$

5. if $u \in V_M$ or $v \in V_M$ then return FALSE

6. $V_M \leftarrow V_M \cup \{u, v\}$

7. return $V_M = V(G)$

WARNING: this takes $\Theta(n)$ time

Perfect Matching Verification

VERIFY-PERFECT-MATCHING(G, M)

1. $V_M \leftarrow \emptyset$ $\triangleright V_M \subseteq V(G)$
2. for each $e \in E(G)$ do
3. if $e \in M$ then
4. $\{u, v\} \leftarrow e$
5. if $u \in V_M$ or $v \in V_M$ then return FALSE
6. $V_M \leftarrow V_M \cup \{u, v\}$
7. return $V_M = V(G)$

WARNING: this takes $\Theta(n)$ time

can you make line 7 take $O(1)$ time?

Perfect Matching Verification

VERIFY-PERFECT-MATCHING(G, M)

1. $V_M \leftarrow \emptyset$ $\triangleright V_M \subseteq V(G)$

2. for each $e \in E(G)$ do

3. if $e \in M$ then

4. $\{u, v\} \leftarrow e$

5. if $u \in V_M$ or $v \in V_M$ then return FALSE

6. $V_M \leftarrow V_M \cup \{u, v\}$

7. return $|V_M| = |V(G)|$

 we have access to set sizes in constant time

Perfect Matching Verification - Running Time

VERIFY-PERFECT-MATCHING(G, M)

1. $V_M \leftarrow \emptyset$ $\triangleright V_M \subseteq V(G)$ } $O(n)$

2. for each $e \in E(G)$ do

3. if $e \in M$ then

4. $\{u, v\} \leftarrow e$

5. if $u \in V_M$ or $v \in V_M$ then return FALSE

6. $V_M \leftarrow V_M \cup \{u, v\}$

7. return $|V_M| = |V(G)|$ } $O(1)$

} $O(1)$ } $O(m)$

Stable Set Verification

Recall: $S \subseteq V(G)$ is stable in a graph G if G has no edge with both ends in S

VERIFY-STABLE-SET(G, S)

1. for each $e \in E(G)$ do
2. $\{u, v\} \leftarrow e$
3. if $u \in S$ and $v \in S$ then return FALSE
4. return TRUE

Stable Set Verification

Recall: $S \subseteq V(G)$ is stable in a graph G if G has no edge with both ends in S

VERIFY-STABLE-SET(G, S)

1. for each $e \in E(G)$ do
 2. $\{u, v\} \leftarrow e$
 3. if $u \in S$ and $v \in S$ then return FALSE
 4. return TRUE
- $\left. \begin{array}{l} \text{2. } \{u, v\} \leftarrow e \\ \text{3. if } u \in S \text{ and } v \in S \text{ then return FALSE} \end{array} \right\} O(1)$
- $\left. \begin{array}{l} \text{2. } \{u, v\} \leftarrow e \\ \text{3. if } u \in S \text{ and } v \in S \text{ then return FALSE} \\ \text{4. return TRUE} \end{array} \right\} O(m)$

Walk Verification

VERIFY-WALK (G, w)

?

Walk Verification

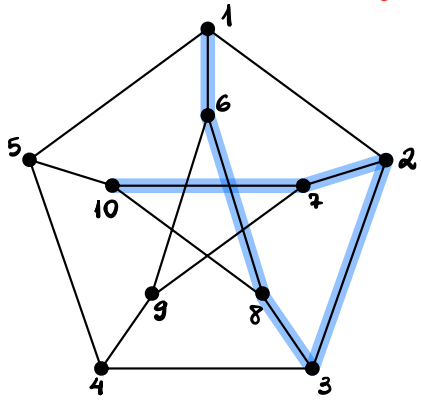
VERIFY-WALK (G, W)

↑
how is this sequence given?

Walk Verification

VERIFY-WALK (G, W)

how is this sequence given?



say, as an array:

indices	→	1	2	3	4	5	6	7
W		1	6	8	3	2	7	10

or

	0	1	2	3	4	5	6
W	1	6	8	3	2	7	10

Walk Verification

VERIFY-WALK (G, W)

1. $\langle v_0, \dots, v_\ell \rangle \leftarrow W$

this binds the symbols $\ell \in \mathbb{N}$ and $v: \{0, \dots, \ell\} \rightarrow V$

Walk Verification

VERIFY-WALK(G, W)

1. $\langle v_0, \dots, v_l \rangle \leftarrow W$
2. for each $i \in [l]$ do
3. ▶ we need to check if $v_{i-1}, v_i \in E(G)$

Walk Verification

VERIFY-WALK (G, W)

1. $\langle v_0, \dots, v_l \rangle \leftarrow W$
2. for each $i \in [l]$ do
3. ▶ we need to check if $v_{i-1}, v_i \in E(G)$

how can we do that in time $O(1)$,

so that the alg. runs in time $O(l)$?

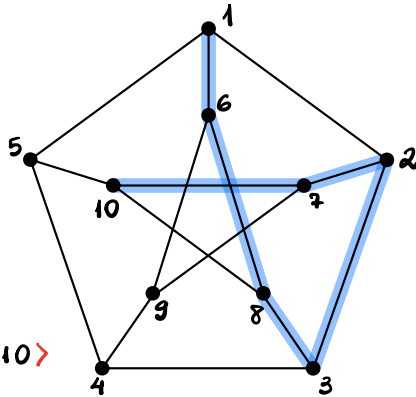
Walk Redefinition

For graphs and digraphs, let's **REDEFINE** walks to be sequences of vertices and edges/arcs alternately, starting and ending with vxs.

Walk Redefinition

For graphs and digraphs, let's **REDEFINE** walks to be sequences of vertices and edges/arcs alternately, starting and ending with vxs.

Example



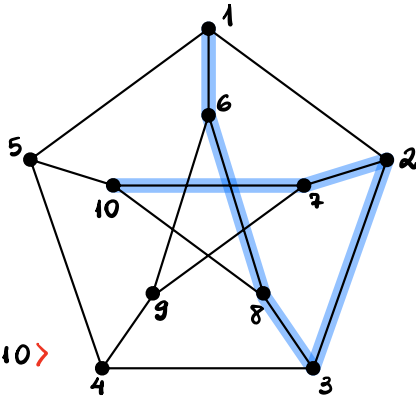
Was: $\langle 1, 6, 8, 3, 2, 7, 10 \rangle$

Is: $\langle 1, \{1,6\}, 6, \{6,8\}, 8, \{8,3\}, 3, \{3,2\}, 2, \{2,7\}, 7, \{7,10\}, 10 \rangle$

Walk Redefinition

For graphs and digraphs, let's **REDEFINE** walks to be sequences of vertices and edges/arcs alternately, starting and ending with vxs.

Example



Was: $\langle 1, 6, 8, 3, 2, 7, 10 \rangle$

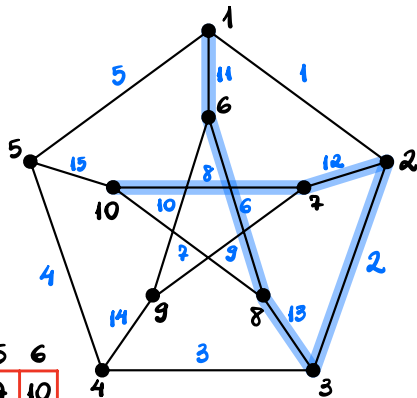
Is: $\langle 1, \{1,6\}, 6, \{6,8\}, 8, \{8,3\}, 3, \{3,2\}, 2, \{2,7\}, 7, \{7,10\}, 10 \rangle$

Abstractly: $\langle v_0, e_1, v_1, \dots, e_\ell, v_\ell \rangle$

Walk Redefinition

For graphs and digraphs, let's **REDEFINE** walks to be sequences of vertices and edges/arcs alternately, starting and ending with vxs.

Example



Was: W

0	1	2	3	4	5	6
1	6	8	3	2	7	10

Is:

W

0	1	2	3	4	5	6	7	8	9	10	11	12
1	11	6	6	8	13	3	2	2	12	7	8	10

Walk Verification

VERIFY-WALK (G, W)

1. $\langle v_0, e_1, v_1, \dots, e_l, v_l \rangle \leftarrow W$
2. for each $i \in [l]$ do
3. $\{u, v\} \leftarrow e_i$

this binds the symbols $l \in \mathbb{N}$, as well as
 $v: \{0, \dots, l\} \rightarrow V$ and $e: [l] \rightarrow E$,
even with the index mismatch w.r.t. W
with respect to

Walk Verification

VERIFY-WALK(G, W)

1. $\langle v_0, e_1, v_1, \dots, e_l, v_l \rangle \leftarrow W$
2. for each $i \in [l]$ do
3. $\{u, v\} \leftarrow e_i$
4. if $\{u, v\} \neq \{v_{i-1}, v_i\}$ then return FALSE
5. return TRUE

Walk Verification

VERIFY-WALK (G, W)

1. $\langle v_0, e_1, v_1, \dots, e_\ell, v_\ell \rangle \leftarrow W$ } $O(\ell)$
 2. for each $i \in [\ell]$ do
 3. $\{u, v\} \leftarrow e_i$
 4. if $\{u, v\} \neq \{v_{i-1}, v_i\}$ then return FALSE } $O(1)$
 5. return TRUE } $O(1)$
- } $O(\ell)$

The Single-Source Shortest Paths Problem

Recall:

Given: a digraph $D := (V, A)$ and a vertex $r \in V$

Find: for each $v \in V$, a shortest rv -path in D

Let $r, s \in V$. An rs -path is **shortest** or **minimum-length** if it is an optimal solution for

Minimize l

subject to $\langle v_0, a_1, v_1, \dots, a_l, v_l \rangle$ is an rs -path in D

updated walk definition

Distance

Let $\mathcal{D} := (V, A)$ be a digraph, and let $r, s \in V$.

Define the distance function $\text{dist}_{\mathcal{D}} : V \times V \rightarrow \mathbb{R} \cup \{+\infty\}$ of $\mathcal{D}$ as follows:

for each $r, s \in V$, define $\text{dist}_{\mathcal{D}}(r, s)$ as

$$\text{dist}_{\mathcal{D}}(r, s) := \begin{cases} \text{the minimum length of an } rs\text{-path} & \text{if } r \rightsquigarrow_{\mathcal{D}} s; \\ +\infty & \text{otherwise} \end{cases}$$

this notation means that
there is an rs -path in $\mathcal{D}$

Distance

Let $\mathcal{D} := (V, A)$ be a digraph, and let $r, s \in V$.

Define the distance function $\text{dist}_{\mathcal{D}} : V \times V \rightarrow \mathbb{R} \cup \{+\infty\}$ of $\mathcal{D}$ as follows:

for each $r, s \in V$, define $\text{dist}_{\mathcal{D}}(r, s)$ as

$$\text{dist}_{\mathcal{D}}(r, s) := \begin{cases} \text{the minimum length of an } rs\text{-path} & \text{if } r \rightsquigarrow_{\mathcal{D}} s; \\ +\infty & \text{otherwise} \end{cases}$$

this notation means that
there is an rs -path in $\mathcal{D}$

if $r \rightsquigarrow_{\mathcal{D}} s$, we say that s is *reachable* from r (in $\mathcal{D}$)

and that r *reaches* s (in $\mathcal{D}$).

Distance

Let $\mathcal{D} := (V, A)$ be a digraph, and let $r, s \in V$.

Define the distance function $\text{dist}_{\mathcal{D}}: V \times V \rightarrow \mathbb{R} \cup \{+\infty\}$ of $\mathcal{D}$ as follows:

for each $r, s \in V$, define $\text{dist}_{\mathcal{D}}(r, s)$ as

as usual, may be omitted...

$$\text{dist}_{\mathcal{D}}(r, s) := \begin{cases} \text{the minimum length of an } rs\text{-path} & \text{if } r \rightsquigarrow_{\mathcal{D}} s; \\ +\infty & \text{otherwise} \end{cases}$$

this notation means that
there is an rs -path in $\mathcal{D}$

if $r \rightsquigarrow_{\mathcal{D}} s$, we say that s is *reachable* from r (in $\mathcal{D}$)

and that r *reaches* s (in $\mathcal{D}$).

Breadth-First Search

alias: BFS

BREADTH-FIRST-SEARCH (D, r)

due to Berge (1958)
and Moore (1959)

1. $Q \leftarrow \emptyset$ ▷ an empty queue
2. for each $v \in V(D)$ do $\pi[v] \leftarrow \text{NIL}$; $d[v] \leftarrow +\infty$
3. $d[r] \leftarrow 0$; ENQUEUE(Q, r)
4. while $Q \neq \emptyset$ do
5. $u \leftarrow \text{FRONT}(Q)$ ▷ access next elt. to leave Q without removing it
6. for each $a \in \mathcal{E}_D^{\text{out}}(u)$ do
7. $(_, v) \leftarrow a$ ▷ alternatively, $v \leftarrow \text{HEAD}(a)$
8. if $d[v] = +\infty$ then
9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; ENQUEUE(Q, v)
10. $_ \leftarrow \text{DEQUEUE}(Q)$ ▷ ignore return value
11. return (π, d)

Breadth-First Search

BREADTH-FIRST-SEARCH ($\mathcal{D}, r$)

1. $Q \leftarrow \emptyset$ ▷ an empty queue
2. for each $v \in V(\mathcal{D})$ do $\pi[v] \leftarrow \text{NIL}$; $d[v] \leftarrow +\infty$
3. $d[r] \leftarrow 0$; ENQUEUE(Q, r)
4. while $Q \neq \emptyset$ do
5. $u \leftarrow \text{FRONT}(Q)$
6. for each $a \in \delta_{\mathcal{D}}^{\text{out}}(u)$ do
7. $(_, v) \leftarrow a$
8. if $d[v] = +\infty$ then
9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; ENQUEUE(Q, v)
10. $_ \leftarrow \text{DEQUEUE}(Q)$
11. return (π, d)

abuse of notation for $\delta_{\mathcal{D}}^{\text{out}}(\{u\})$

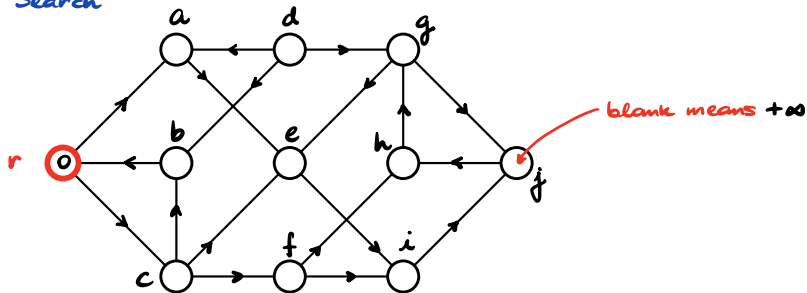
Breadth-First Search

BREADTH-FIRST-SEARCH ($\mathcal{D}, r$)

An important cousin of BFS called DFS is typically covered in the course
MAC0328 Graph Algorithms

1. $Q \leftarrow \emptyset$ $\triangleright$ an empty queue
2. for each $v \in V(\mathcal{D})$ do $\pi[v] \leftarrow \text{NIL}$; $d[v] \leftarrow +\infty$
3. $d[r] \leftarrow 0$; ENQUEUE(Q, r)
4. while $Q \neq \emptyset$ do
5. $u \leftarrow \text{FRONT}(Q)$
6. for each $a \in \mathcal{E}_{\mathcal{D}}^{\text{out}}(u)$ do
7. $(_, v) \leftarrow a$
8. if $d[v] = +\infty$ then
9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; ENQUEUE(Q, v)
10. $_ \leftarrow \text{DEQUEUE}(Q)$
11. return (π, d)

Breadth-First Search



Q r

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

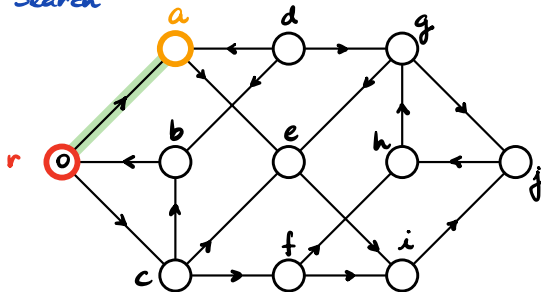
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q r

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

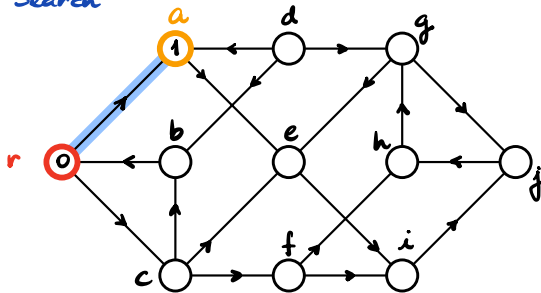
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

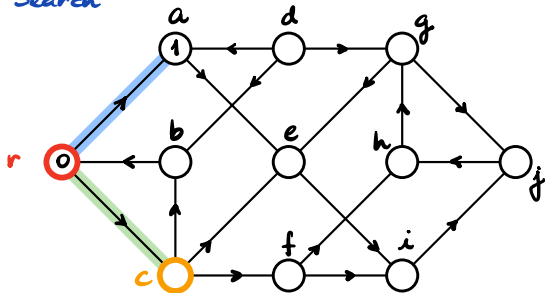
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; ENQUEUE(Q, v)

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

r	a
-----	-----

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

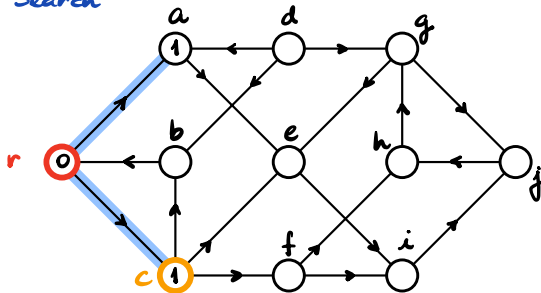
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

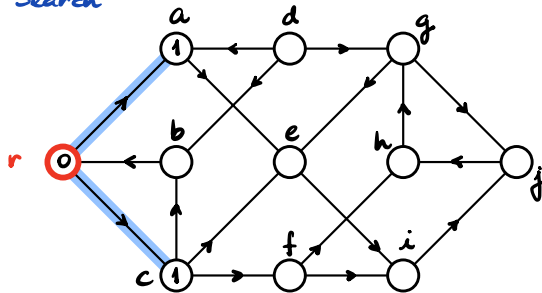
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; ENQUEUE(Q, v)

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

a	c
---	---

r

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

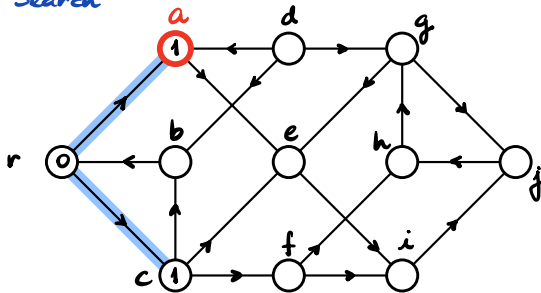
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

a	c
---	---

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

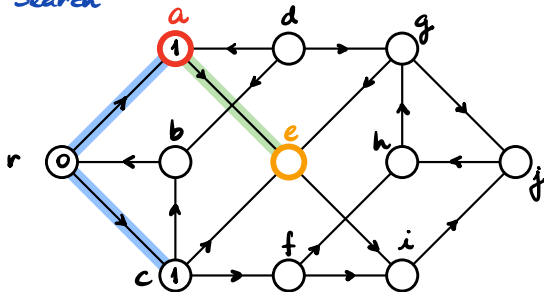
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

a	c
---	---

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

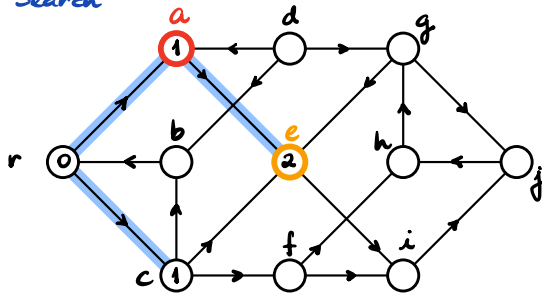
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

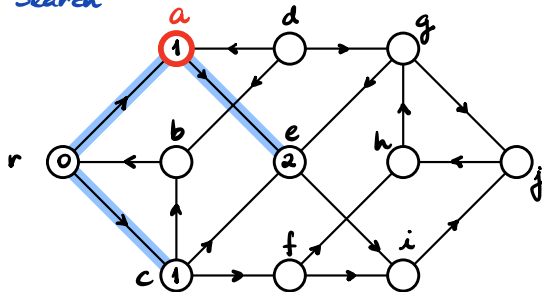
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; ENQUEUE(Q, v)

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

c	e
---	---

a

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

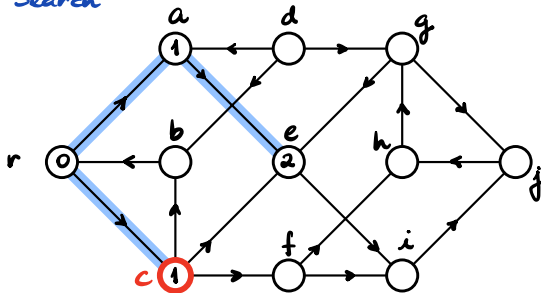
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

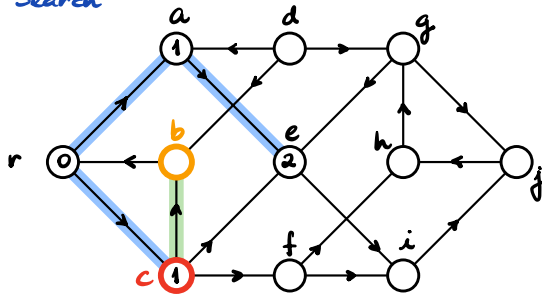
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

c	e
---	---

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

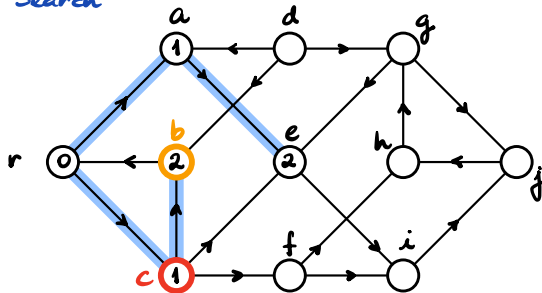
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q [c | e | b]

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

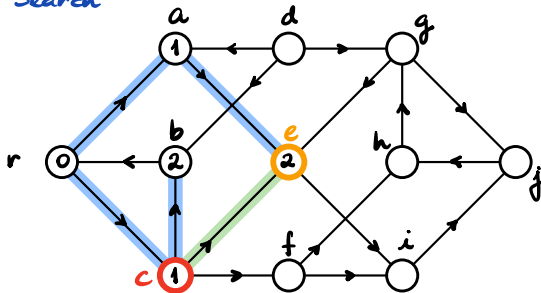
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; ENQUEUE(Q, v)

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

c	e	b
---	---	---

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

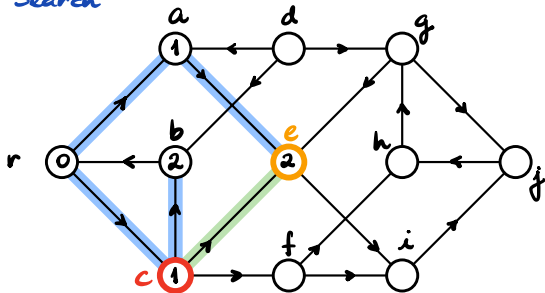
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

c	e	b
---	---	---

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

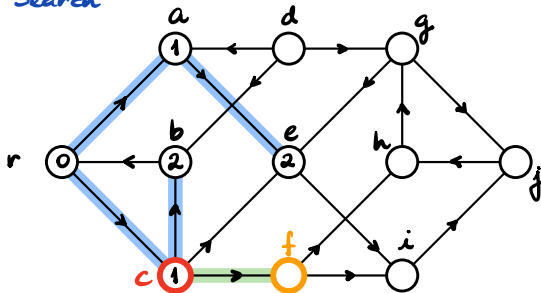
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; ENQUEUE(Q, v)

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

c	e	b
---	---	---

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

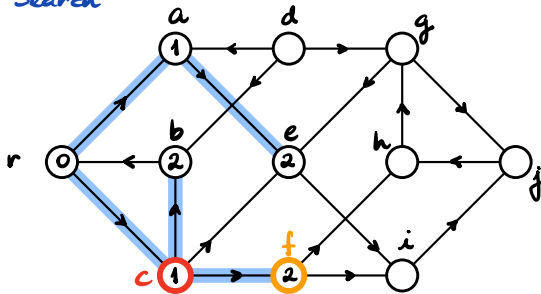
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

c	e	b	f
---	---	---	---

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

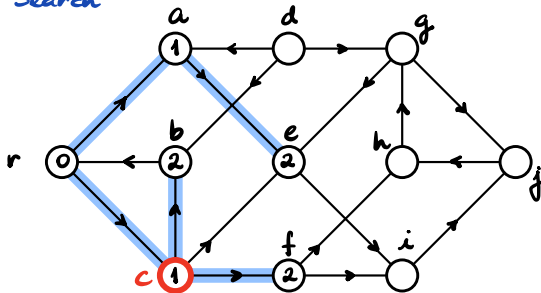
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; ENQUEUE(Q, v)

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

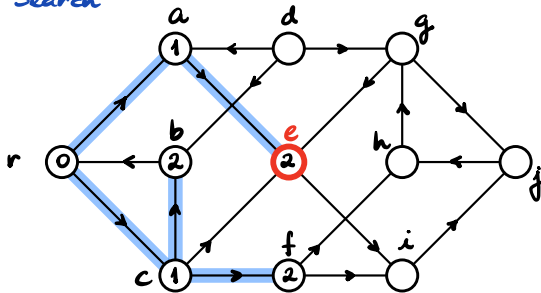
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

e	b	f
---	---	---

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

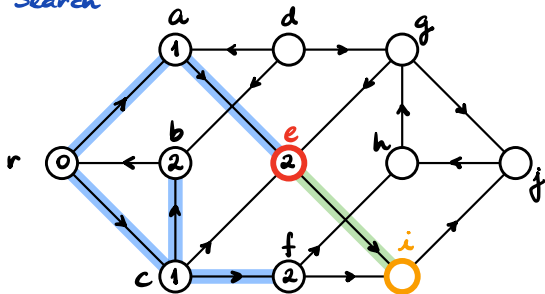
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; ENQUEUE(Q, v)

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

e	b	f
---	---	---

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

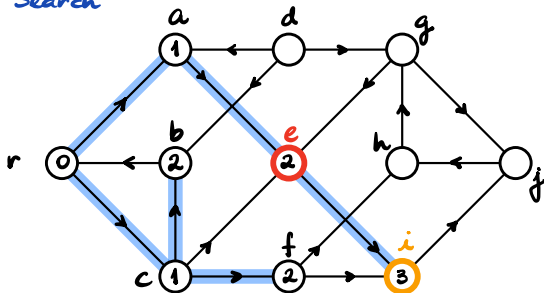
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

e	b	f	i
---	---	---	---

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

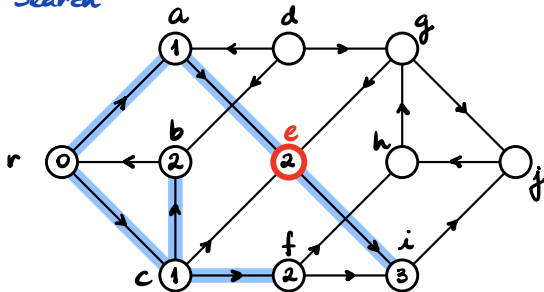
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; ENQUEUE(Q, v)

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

b	f	i
---	---	---

e

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

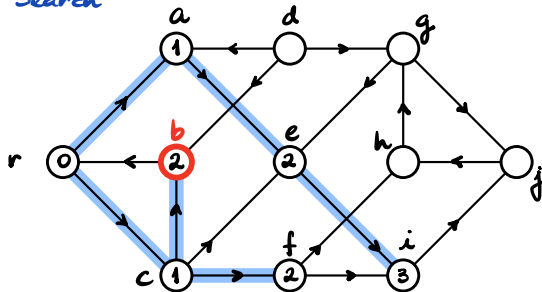
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; ENQUEUE(Q, v)

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

b	f	i
---	---	---

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

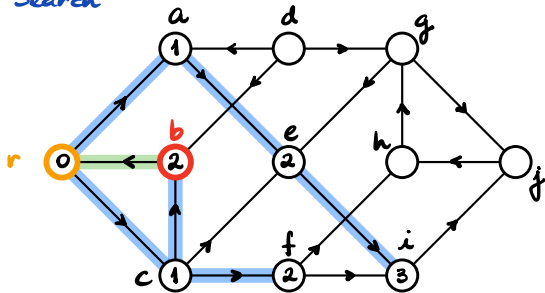
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; ENQUEUE(Q, v)

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q [b | f | i]

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

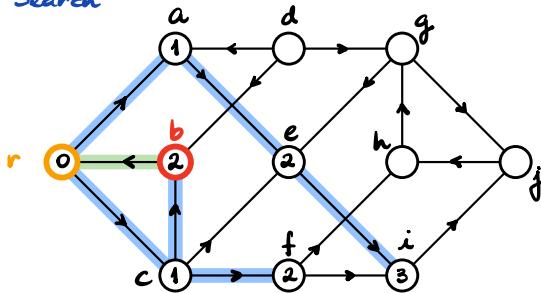
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; ENQUEUE(Q, v)

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

b	f	i
---	---	---

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

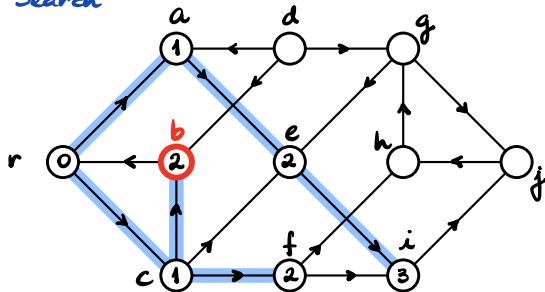
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

f	i
---	---

b

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

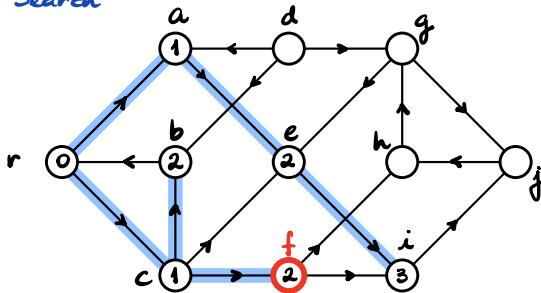
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

f	i
---	---

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

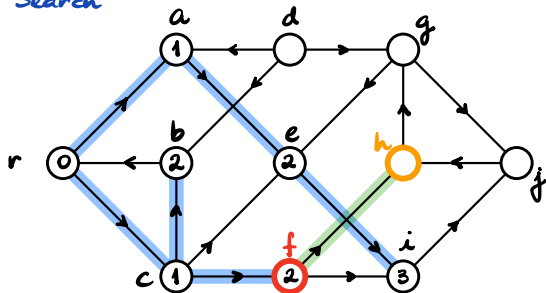
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q f i

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

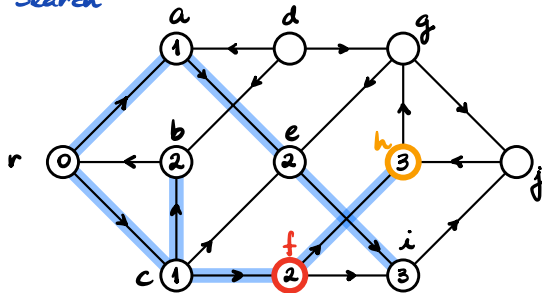
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

f	i	h
---	---	---

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

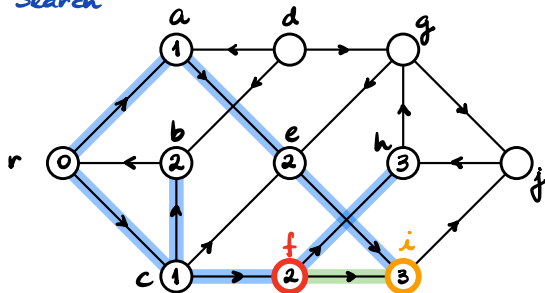
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; ENQUEUE(Q, v)

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

f	i	h
---	---	---

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

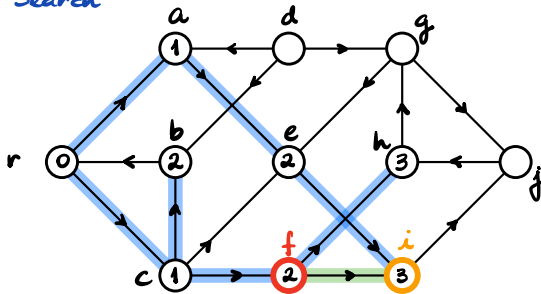
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

f	i	h
---	---	---

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

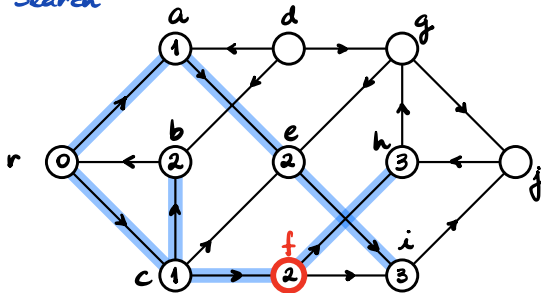
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

i	h
-----	-----

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

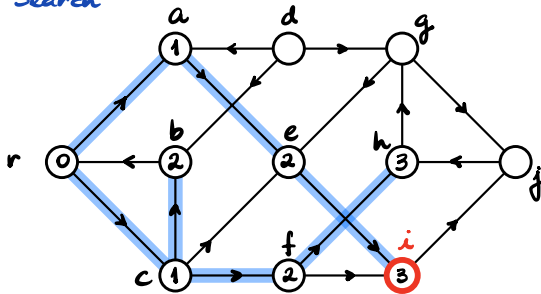
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

i	h
-----	-----

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

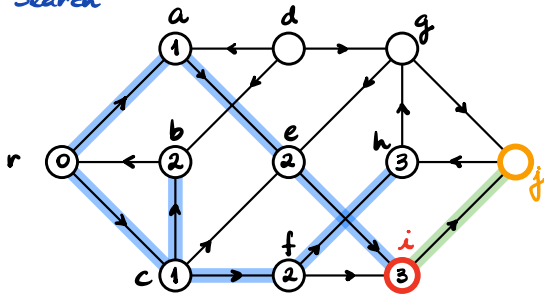
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

i	h
-----	-----

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

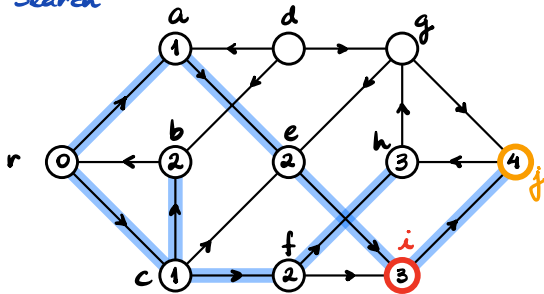
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

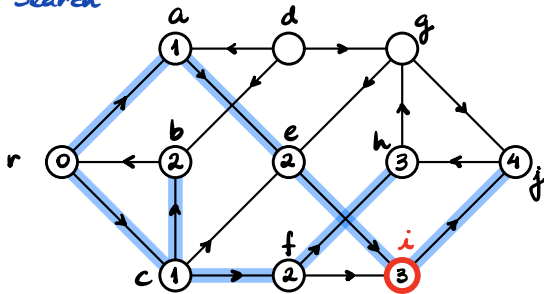
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; ENQUEUE(Q, v)

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

h	j
----------	----------

i

4. while $Q \neq \emptyset$ do

5. **u** ← FRONT(Q)

6. for each $a \in \delta^{\text{out}}(u)$ do

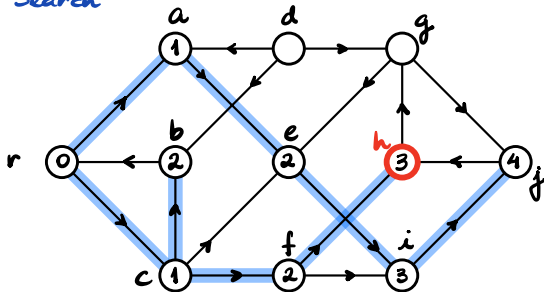
7. (**_**, **v**) ← a

8. if $d[\textbf{v}] = +\infty$ then

9. $\pi[\textbf{v}] \leftarrow a$; $d[\textbf{v}] \leftarrow d[\textbf{u}] + 1$; ENQUEUE(Q , **v**)

10. **_** ← DEQUEUE(Q)

Breadth-First Search



4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

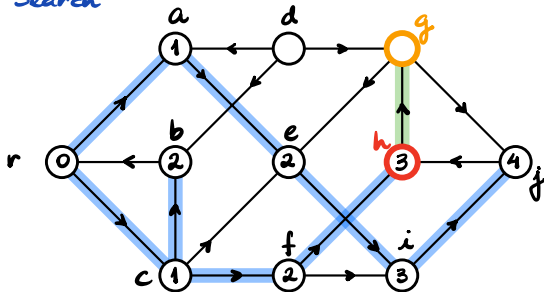
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

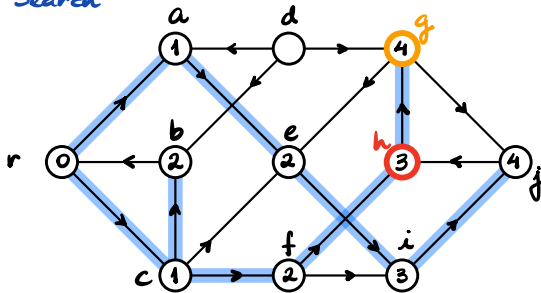
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

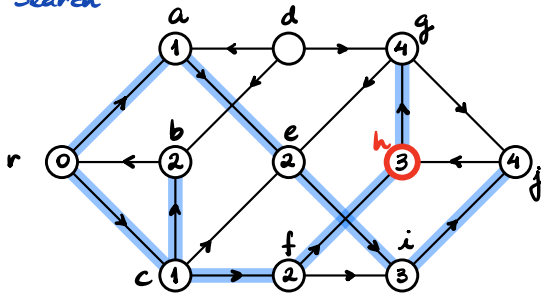
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; ENQUEUE(Q, v)

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q

j	g
---	---

h

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

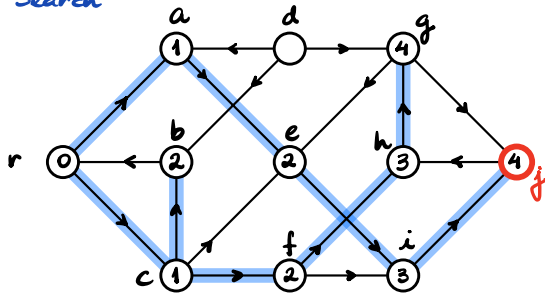
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q f g

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

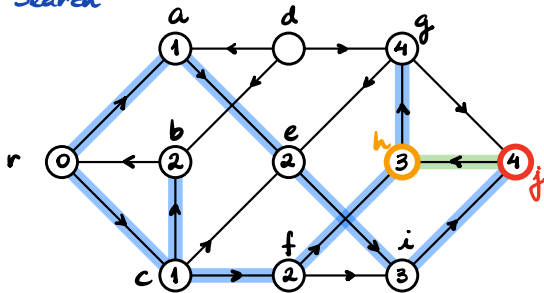
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

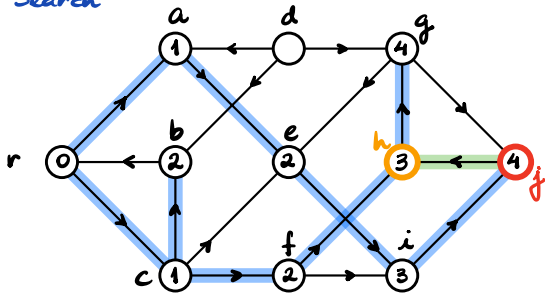
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

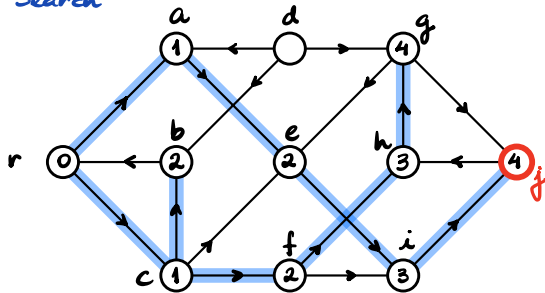
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

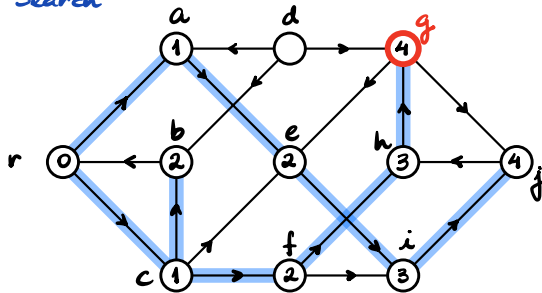
10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



4. while $Q \neq \emptyset$ do
5. $u \leftarrow \text{FRONT}(Q)$
6. for each $a \in \delta^{\text{out}}(u)$ do
7. $(_, v) \leftarrow a$
8. if $d[v] = +\infty$ then
9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$
10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q g

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

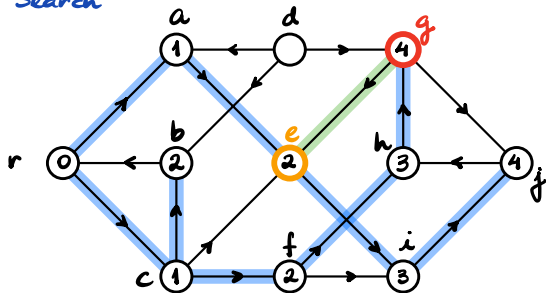
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q g

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

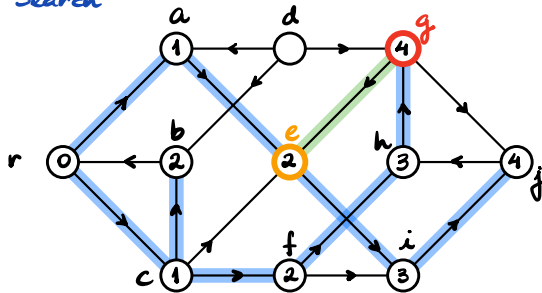
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q g

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

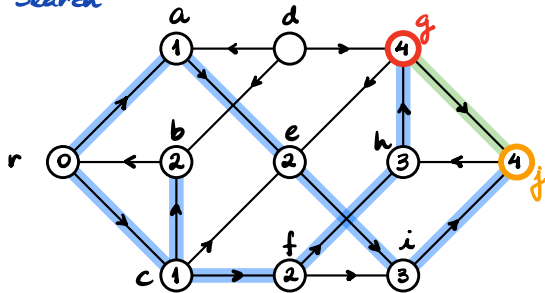
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q g

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

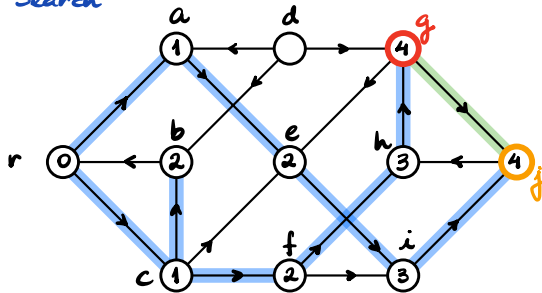
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



Q g

4. while $Q \neq \emptyset$ do

5. $u \leftarrow \text{FRONT}(Q)$

6. for each $a \in \delta^{\text{out}}(u)$ do

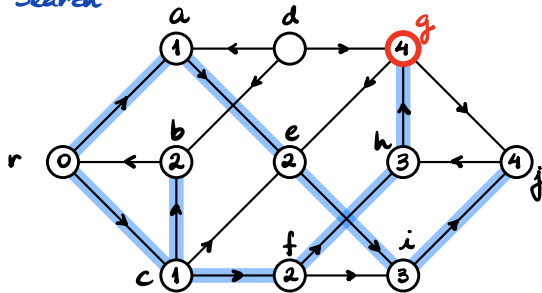
7. $(_, v) \leftarrow a$

8. if $d[v] = +\infty$ then

9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$

10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search



4. while $Q \neq \emptyset$ do
5. $u \leftarrow \text{FRONT}(Q)$
6. for each $a \in \delta^{\text{out}}(u)$ do
7. $(_, v) \leftarrow a$
8. if $d[v] = +\infty$ then
9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; $\text{ENQUEUE}(Q, v)$
10. $_ \leftarrow \text{DEQUEUE}(Q)$

Breadth-First Search

BREADTH-FIRST-SEARCH ($\mathcal{D}, r$)

1. $Q \leftarrow \emptyset$ ▷ an empty queue
2. for each $v \in V(\mathcal{D})$ do $\pi[v] \leftarrow \text{NIL}$; $d[v] \leftarrow +\infty$
3. $d[r] \leftarrow 0$; ENQUEUE(Q, r)
4. while $Q \neq \emptyset$ do
5. $u \leftarrow \text{FRONT}(Q)$ ▷ access next elt. to leave Q without removing it
6. for each $a \in \mathcal{E}_{\mathcal{D}}^{\text{out}}(u)$ do
7. $(_, v) \leftarrow a$ ▷ alternatively, $v \leftarrow \text{HEAD}(a)$
8. if $d[v] = +\infty$ then
9. $\pi[v] \leftarrow a$; $d[v] \leftarrow d[u] + 1$; ENQUEUE(Q, v)
10. $_ \leftarrow \text{DEQUEUE}(Q)$ ▷ ignore return value
11. return (π, d)

"Obvious" BFS Properties

- (1) Each $v \in V$ is added to Q at most once, precisely if $d[v]$ becomes finite; in that case, v eventually leaves Q and it is never again added to Q .
- (2) The while loop terminates after $\leq n$ iterations.
- (3) Every vertex $v \in V$ with finite $d[v]$ has $\pi[v] \neq \text{NIL}$, except for r .
- (4) If $v \in V$ has a non-NIL $\pi[v] =: a$, then $a \in A$ is an arc with v as its head, and its tail u has $d[u] + 1 = d[v] < +\infty$.